ChatGPT

AI Control Plane & State Admission

The system (nicknamed “Brudo/Grax”) enforces a mandatory **control plane** between LLM reasoning and execution【30†L59-L67】【30†L97-L104】. In practice, it reconstructs a candidate system state from logs but today *implicitly trusts* it. We propose inserting a *typed admission engine* here: before the state becomes authoritative, it must pass explicit validation predicates (authority, temporal, tracking, etc.) with fail-closed semantics. This is analogous to “execution gates” in AI governance. For example, one demonstration system requires that **every action pass a pre-execution gate stack** before running【33†L48-L56】 (fail-closed by default【33†L38-L46】). Our engine would similarly block any state that fails its typed checks, preventing “uneared” state from entering the runtime.

Partial Observability & Consequence Identifiability

Classical **runtime verification** of partial traces already uses three-valued logic (true/false/inconclusive) to handle unknown or incomplete observations【10†L90-L99】. We generalize this: instead of a blind “inconclusive”, we form the **observation-equivalence class** of all compatible worlds. Then for a given consequence or transition C, we derive its value independently in each world. If all worlds in the class agree on C, we can *admit* execution (IDENTIFIABLE); if they disagree, we must *withhold* (NOT_IDENTIFIABLE). In effect, *identifiability is indexed to each consequence*. The same observed state might decisively fix one transition but leave another ambiguous. Only when $|\{C(R) : R \in E\}| = 1$ do we allow C to execute. Otherwise we withhold, not by random guess, but as long as distinct worlds imply different lawful outcomes. (This guards against implicitly collapsing world uncertainty into a false execution.)

Counterfactual Independence Testing

To ensure no circular logic, we propose a **counterfactual assay** on the dependency basis. Whenever a consequence is derived from a basis B and a proposed label/outcome L, we test for *independence* by intervention. Concretely, we “swap” or alter L and recompute C on the same B. If intervening on L changes the resulting C, then the basis was improperly using the outcome itself – a circular dependency – and independence is not proven. In other words, we build a structured **IndependenceReceipt** that records the derivation tree, the intervention (e.g. label swap), the recomputed outcomes, and whether they match. This is akin to Pearl’s do-calculus: we intervene on the outcome to see if it propagates backwards. (Such counterfactual tests are common in causality/fairness research, although here the novelty is their use as a pre-execution gate.)

Active Learning & Minimum Observations

When a consequence is non-identifiable, we turn the problem into an **active learning/experiment design** task. We have a “version space” of worlds E, and need to pick a next observation (question) to split that space. The optimal choice is the query whose answers would maximally reduce *consequence uncertainty*.

Formally, choose query q that maximizes expected information gain: the reduction in entropy of the distribution of remaining C values per unit cost【18†L120-L127】. Expected Information Gain is a standard criterion in sequential design and active learning【18†L120-L127】. In practice, we compute for each candidate observation q the weighted entropy of the partitioned classes $E_{q=v}$ and pick the query that most strongly distinguishes the differing outcomes. Crucially, we also allow a **stop option**: if the expected gain is lower than the cost, the engine should opt **DO_NOT_OBSERVE** (cease querying) and remain in WITHHOLD. This aligns with decision-theoretic value-of-information principles: don't ask a question unless its benefit justifies the effort. (In turn, a complementary "minimum sufficient observation" routine could prune away already-resolved observations to compress the evidentiary basis without losing determinacy.)

Proposed Admission Pipeline Architecture

Taken together, the design defines a *pipeline of admission questions*: first **State Admission** (does the past state pass all typed predicates?), then **Derivation Admission** (was this consequence derived without circular dependency?), and finally **Consequence Admission** (is the future outcome invariant across the admitted world class?). In short:

- **State Admission (A_S):** Candidate state → Typed predicates (BYTE_IDENTITY, AUTHORITY, TEMPORAL_SCOPE, TRACKED_STATE, CANON_ELIGIBILITY, etc.) → Authoritative state or reject. Fail-closed on any predicate failure.
- **Derivation Admission (A_D):** Given basis B for transition, test that B does not (causally) depend on the proposed outcome. Produce a *ConsequenceBasis* and *IndependenceReceipt* encapsulating dependencies and intervention results.
- **Consequence Admission (A_C):** Given admitted state and desired consequence C, compute C on all worlds in the current observation class and collect outcomes. If all outcomes are identical, commit the transition; if more than one outcome appears, withhold execution as NOT_IDENTIFIABLE (until further observation).

This is a true **fail-closed gate stack** before execution【33†L38-L46】. It separates byte/format integrity (already verified) from semantic authority. Only after passing every gate does a state or action become "authoritative." Such a structured pipeline is reminiscent of the "AI control plane" architecture, where the model only proposes actions and an independent plane enforces them【30†L59-L67】【30†L97-L104】.

Patent Landscape & Prior Work

There is growing recognition of the need for such pre-execution gates. For example, "AI control plane" vendors explicitly separate reasoning and execution, enforcing that model proposals be checked by rules 【30†L59-L67】【30†L97-L104】. In the patent domain, one framework ("Execution Gate Orchestration System") describes multi-layer gates (risk scoring, safe-mode write-blocking, etc.) that all must pass *before* any action runs【33†L48-L56】【33†L38-L46】. These concepts align with our pipeline: a mandatory decision boundary that intercepts model intent and applies deterministic checks.

However, we found no prior art that combines *typed admission predicates*, *independent consequence derivation*, and *invariant-consequence gating* as a cohesive architecture. Generic structured-output validation (e.g. JSON schema checking) is known (see, e.g. JSON schema patents【42†L0-L2】), but those do **not** bind

the schema identity into the job identity or the execution decision. Similarly, runtime monitors with inconclusive outcomes are known【10†L90-L99】, but not as a gate for execution decisions by consequence. Our approach explicitly ties:
"WorkIdentity ↔ ContractIdentity ↔ ResultIdentity ↔ AdmissionReceipt ↔ PermittedConsequence." This tight binding of intent (contract) to authorization (gate) appears novel.

Potential Patentable Claims

Based on this analysis, key claim candidates include:

- **Typed State Admission Engine:** A computer-implemented method that transforms a reconstructed candidate state into authoritative state only after evaluating a *fixed set of typed predicates* (authority, temporal, tracking, etc.), with fail-closed semantics. (Distinguish from legacy systems that assume restored state is correct.)
- **Consequence-Indexed Execution Gate:** A system that, for a given observation class, derives the downstream transition consequence independently on all compatible worlds and **permits execution only if all consequences agree**. Invariants over the class are collapsed; non-invariant transitions are withheld.
- **Dependency-Resistant Derivation Receipt:** Rather than a simple Boolean, produce a structured receipt (proof) that a consequence derivation is independent of its own proposed outcome. This includes recording the derivation basis, any interventions (e.g. label swaps), recomputed results, and a computed independence status.
- **Active Disambiguation Selector:** A mechanism that computes the next optimal observation to reduce consequence ambiguity. For example, selecting a query to maximize expected information gain (entropy reduction) over remaining worlds, or finding a minimal subset of existing observations sufficient to decide a consequence. (Incorporates cost-aware stopping.)
- **Typed Admission Receipts and Non-Amplification:** Logging each admission decision in a replayable receipt that records exactly which predicates passed or failed, binding each result to the original contract identity. Ensuring that replaying the same output under the same contract cannot yield a more permissive outcome (no "stronger admission" on repeat).

Each of these builds on existing ideas but in new combinations: tying generative result verification to a fixed contract identity and a formal control gate, and using information-theoretic selection on nondeterministic outputs. Prior art on AI gates, schema validation, or uncertainty gating (e.g. uncertainty-based LLM routing) might be considered, but none appear to claim *executable admission gates* driven by consequence invariance and typed predicates. We will examine patents in AI governance and control (e.g. execution gate patents, AI pipeline enforcement, data pipeline security) to find the closest references.

Implementation Gaps & Next Steps

Currently, the repository has low-level verification and static checks, but lacks the dynamic admission machinery. The immediate engineering steps are: (1) replace implicit trust of restored state with an explicit admission API; (2) implement the typed predicate engine (with receipts for each check); (3) implement the consequence-equivalence check and gating; and (4) build the query-selection optimizer (information gain). Throughout, **fail-closed** behavior and structured receipts are critical: unknown predicates should produce a WITHHOLD rather than guess.

In summary, this line of work reframes “making the AI system smarter” into a series of *deterministic, testable filters* on its outputs. It draws on concepts from runtime verification【10†L90-L99】, causal intervention, and active learning【18†L120-L127】, but applies them specifically to enforce safe execution boundaries. That strongly suggests a technical, patentable improvement to AI infrastructure rather than abstract AI governance, fitting within USPTO guidance for software improvements. The most promising claims will likely focus on the concrete admission mechanisms (typed predicates, consequence gates, receipts) as a coherent system, all supported by our prototype implementation.

Sources: We draw on runtime-monitoring research【10†L90-L99】, information-theoretic experiment design【18†L120-L127】, and industry analyses of AI control planes【30†L59-L67】【30†L97-L104】 and execution gating【33†L48-L56】【33†L38-L46】. These illustrate the technical context and support the novelty of our proposed admission framework.
